Design notes

Why the method is shaped the way it is: the decisions made while building `dl-prototype`, what was taken from other spec-driven systems and what was left behind, and what is still open. Newest decisions are at the end of each section; open items are at the bottom.

---

1. GitHub Spec Kit

Spec Kit is GitHub's open-source toolkit for spec-driven development. It runs a chain of commands (constitution, specify, clarify, plan, analyze, tasks, implement) and writes plain Markdown into the repo. It installs into Claude Code as native skills.

Assessment for games. It runs inside Claude Code without trouble, but it was built for software features: its templates expect data models and API contracts, its flow runs one way from specify to implement, and it gives little help for changing a spec mid-build. Reviewers report a large volume of documents to review, and specs only govern by convention: nothing checks the build against them.

Taken

Idea	Where it lives in this method
The constitution as one file every agent reads first	`templates/constitution.md`, `constitution` and `adopt` skills
Clarify as a gate	`clarify` skill; no Ready with a blocking open question
Analyse as a cross-artifact consistency check	`analyze` skill, run as a separate agent
A short path and a full path	`brief` versus `spec`
Tasks handed to tickets	`tasks` skill, in the job story and Gherkin format
Layered overrides (project over extension over core)	`specs/overrides/templates/` beats the plugin's templates
Portability across agents	Core rules in `AGENTS.md`, with `CLAUDE.md` pointing to it
Separate starts for new and existing projects	`constitution` and `adopt`

Left behind: the Python command-line install; software templates (data model, contracts, research); the one-way specify-plan-tasks order; autonomous implementation.

---

2. Proviso

Proviso, by Bruno Baketarić, keeps one spec tree true across many changes. Every change is an amendment on a branch; a command-line tool diffs it against a baseline and lists what it might affect; separate agents build, test and check it without seeing each other's work. Its own documentation says it trades speed for coherence and is overhead for prototypes. Licensed Apache 2.0.

Taken into the prototype tier now

Idea	Where it lives
Guarded rules: When, Provided, Then, Otherwise, with examples on both sides	`principles/04-writing-specs.md`, feature spec template
`?` marks an undecided place, and nothing with a blocking `?` is built	Open questions convention, Ready gate
Who decides what; agents decide nothing and report gaps	`principles/02-who-decides.md`
A checking agent that sees no code, tickets or playtests	`analyze` skill
A plain summary for the business owner, with open questions as concrete cases	`spec` and `clarify` skills
An amendment note saying what is deliberately left out	"Not in this brief", "Not ticketed"
A design boundary: what the game relies on but does not build	Constitution, "Platforms and boundary"
Kit owned by the tool, notes owned by the project	Plugin versus `specs/`, with overrides

Saved for the production tier: a baseline and manifest with mechanical diff and impact lists; context packs that give each agent only what it needs; independent testing and coding agents in separate worktrees; mutation checks on the tests; a request skill for non-technical clients that runs in their own assistant; verification in CI.

Where it does not fit games: feel and visual work cannot be written as guarded rules; black-box testing through contracts suits a back end far better than a Unity UI; and a spec built to stay stable fights a design still being discovered.

---

3. Two tiers

Decision: one method, two plugins. `dl-prototype` is light and revises specs after every playtest. `dl-production` (planned, modelled on Proviso) keeps rules true

across many changes. A feature graduates when a wrong rule costs money or trust, when other features keep breaking as it changes, when it is live and changed by people who did not design it, or when playtests only change its tuning values. Feel and visual work never graduates.

To keep graduation cheap, the prototype tier already uses guarded rules, stable rule IDs and a Parameters table.

---

4. Team model

Decision: the default is one lead agent with at most two helper agents, each on one briefed task and never on the same files. It is the lighter of two models tried in practice. A named team of specialist agents behind an orchestrator also works, but it costs more to set up and tends to let agents make design calls. Teams can add specialists on top; the "who decides" rules still apply.

---

5. Two starts and the adopt trial

Decision: a new game starts with `constitution`; an existing game starts with `adopt`, which reads before it asks and changes nothing that exists. On an existing game, a feature gets a spec only when it is next changed (spec the change, not the game), and when a document and the build disagree, the build wins and the disagreement becomes an open question.

`adopt` was trialled on a real mobile game in production with about 130 design documents and an open questions register of around 270 rows. The trial changed the skill in five ways:

1. **Ask which register is master before importing.** Two live open-question registers drift apart within days.
2. **Keep existing question IDs** and group them by screen or area when the old register has no sections.
3. **Triage large imports first.** 170 open questions with no Blocking flag is more than a team can act on.

4. **Ask for code or build access up front.** Without it, every document's trust stays Unknown.
5. **Look for setup rules that block the method,** such as a rule forbidding cross-document consistency checks, agents that make design decisions, or ticket projects named differently in different documents.

Open

- **Licence and ownership.** The repository is private and all rights are reserved until a licence is chosen and ownership (personal or studio) is settled.
- **The production tier.** Not yet designed beyond the list in section 2.
- **The method flow diagram** shows the loop but not the two starts; the loop diagram does.
- **Copies on claude.ai.** The method guide, worked example and cheat sheet also exist as claude.ai artifacts. This repository is the source of truth; the artifacts may lag behind it.